

Q1. What is the value of result?

python

```
result = 2 ** 3 ** 2 + 10 // 3 * 2 - 4 % 3
```

- A) 512
- B) 517
- C) 519
- D) 522

Q2. Evaluate this bitwise-logical hybrid:

python

```
x = 5
```

```
y = 2
```

```
z = 3
```

```
print(x & y | z ^ x << 1)
```

- A) 9
- B) 13
- C) 15
- D) 7

Q3. What does the following print?

python

```
print(not 5 > 3 & 2 or 4 ** 2 // 3 + ~1 & 7)
```

- A) False
- B) True
- C) 11
- D) 12

Q4. What is printed?

python

```
a = 256
```

```
b = 257
```

```
print((a is b - 1) == (a == b - 1))
```

- A) True
- B) False
- C) TypeError
- D) SyntaxError

Q5. What is the final value of res?

python

res = ~(5 + 3) << 2 & 7 or not 10 // 3

- A) 4
- B) 0
- C) True
- D) 1

Q6. What is the type and value of result?

python

result = 0 or "Python" and [] or 42

- A) 42 (int)
- B) [] (list)
- C) "Python" (str)
- D) True (bool)

Q7. How many times is lst.pop() executed, and what is the final value of result?

python

lst = [10, 20, 30, 40]

x = 0

result = (x > 0) and (lst.pop() > 15) or (lst.pop() < 25)

- A) 1 call, True
- B) 2 calls, True
- C) 1 call, False
- D) 2 calls, False

Q8. What is printed?

python

print(5 or 0 and 2, 0 and 5 or 2)

- A) 5 2
- B) True True
- C) 5 0
- D) 0 2

Q9. Why does this loop never terminate, and what is the fix?

python

i = 0

while i < 10:

if i % 3 == 0:

continue

print(i)

i += 1

A) i is never incremented when i%3==0; fix by moving i += 1 before the if.

B) The condition i < 10 is always true; fix by changing to <=.

C) continue is invalid inside while.

D) print(i) raises an error.

Q10. Which of the following expressions does NOT raise a ZeroDivisionError?

A) 10 // 0 or 5

B) 0 or 10 // 0

C) 0 & 10 // 0

D) 10 // 0 and 5

Q11. What is the output of:

python

print(-17 // 4, -17 % 4, 17 // -4, 17 % -4)

A) -5 3 -5 -3

B) -4 -1 -4 1

C) -5 3 -4 1

D) -4 3 -5 -3

Q12. Compute ~-5 and ~5 respectively:

A) 4, -6

B) -4, 6

C) 4, 6

D) -4, -6

Q13. What is the integer output of:

python

print((~0b1010) & 0b1111, (-5 >> 1) & 0b111)

A) 5, 3

B) 5, 5

C) 10, 3

D) 10, 5

Q14. Verify the Euclidean identity: $a = b * (a // b) + (a \% b)$.

Which of the following is FALSE for $a = -10$, $b = 3$?

- A) $a // b$ is -4
- B) $a \% b$ is 2
- C) $3 * -4 + 2 = -10$
- D) $a // b$ is -3

Q15. What is printed?

python

`print(True + True, False * 3, ~False & True)`

- A) 2 0 1
- B) 2 0 0
- C) 1 0 1
- D) 2 3 0

Q16. What is printed?

python

`lst = [1, 2]`

`print(1 in lst == True)`

- A) True
- B) False
- C) TypeError
- D) ValueError

Q17. How many times is `pop()` called in this chain?

python

`lst = [1, 2, 3, 4]`

`print(lst.pop() < lst.pop() < lst.pop())`

- A) 1
- B) 2
- C) 3
- D) 4

Q18. (Memory interning) What is printed?

```
python
a = 1000
b = 1000
c = a
d = a + 0
print(a is b, a is c, a is d)
```

- A) False True False
- B) False True True
- C) True True True
- D) False False False

Q19. What is printed?

```
python
s = "abcabc"
print("ab" in s, "ab" in s[1:], s[1:] is s)
```

- A) True True False
- B) True False False
- C) False True False
- D) True True True

Q20. (Chained comparison with mutation) What is printed and what is the final lst?

```
python
lst = [10, 5, 0]
print(1 < lst.pop() < 6)
print(lst)
```

- A) True [10, 0]
- B) False [10, 0]
- C) True [10, 5]
- D) False [10, 5]

Q21. What is the output of this loop?

```
python
for i in range(3):
    if i == 1:
        continue
    print(i, end=" ")
else:
    print("Done")
```

- A) 0 2 Done
- B) 0 1 2 Done
- C) 0 2
- D) 0 1 2

Q22. What is the total value of count after execution?

```
python
count = 0
i = 0
while i < 3:
    j = 0
    while j < 3:
        count += 1
        if j == 1:
            break
        j += 1
    i += 1
print(count)
```

- A) 3
- B) 6
- C) 9
- D) Infinite loop

Q23.

Assertion (A):

a = []

b = a

x = a or b and [1]

a.append(1)

print(x is a)

print(x == a)

Reason (R):

and has higher precedence than or, and a or (b and [1]) evaluates to the first truthy operand without necessarily evaluating every operand.

- A. Both A and R are true, and R correctly explains A
- B. Both A and R are true, but R does not correctly explain A
- C. A is true, but R is false
- D. A is false, but R is true

Q24.

Assertion (A):

x = 5

result = 1 < x < 10 == x

is equivalent to:

result = (1 < x) < (10 == x)

Reason (R):

Python's chained comparison:

a < b < c

is evaluated as:

(a < b) and (b < c)

and b is evaluated only once.

- A. Both A and R are true, and R correctly explains A
- B. Both A and R are true, but R does not correctly explain A
- C. A is true, but R is false
- D. A is false, but R is true

Q25.

Assertion (A):

x = 6

y = 3

print(x & y == 2)

prints:

True

Reason (R):

Bitwise & has higher precedence than comparison ==, so Python evaluates:

(x & y) == 2

- A. Both A and R are true, and R correctly explains A
- B. Both A and R are true, but R does not correctly explain A
- C. A is true, but R is false
- D. A is false, but R is true